3.4 Attributes and Override

This tutorial supplements the book. Attributes arrived in PHP 8.0; the built-in method-level Override attribute arrived in PHP 8.3. Our course target is PHP 8.4.

What is an attribute?

An attribute attaches structured metadata to a declaration. Tools or PHP itself can read that metadata and act on it.

The syntax is `#[AttributeName]`, optionally with arguments, placed before the declaration it describes. Attributes can annotate classes, methods, functions, properties, parameters, and constants. A framework might read routing metadata through reflection. Attaching metadata does not, by itself, implement the behavior described. Native attributes have PHP-defined meanings. See the PHP attributes overview.

Distinguish three uses of "attribute" students may encounter:

- A domain object's attribute, such as a food's name, is represented by a property.
- A PHP attribute is metadata written with `#[...]`.
- A comment such as `@return` is a documentation annotation; it is not native attribute syntax.

`#[Attribute]` marks a class as an attribute definition. It is not the generic prefix used whenever applying an attribute. We will use an existing attribute, so no custom attribute class or reflection code is needed.

Declare the intent to override

Run this complete example in `3-4-override-attribute.php`:

```php
<?php
declare(strict_types=1);

class Message
{
    public function getText(): string
    {
        return 'Hello';
    }
}

class WelcomeMessage extends Message
{
    #[Override]
    public function getText(): string
    {
        return 'Welcome to class';
    }
}
```

```
echo (new WelcomeMessage())->getText(), PHP_EOL;
```

Output: `Welcome to class`.

We are in the global namespace, so `#[Override]` resolves to PHP's built-in attribute. In a named namespace, import it with `use Override;` before using the short name. You may also encounter `#[\Override]`, which explicitly names the global class. Namespaces are next week's topic.

Make the typo visible

1. Rename **only** the child method to `getTetx()`; leave the call and attribute unchanged.
2. Run the script. PHP emits a compile-time error because the marked method does not override a matching parent method.
3. Remove the attribute while keeping the typo. Run again: the inherited `getText()` returns `Hello`. The misspelled method merely became a new, unused method.
4. Restore `getText()` and the attribute. The intended output returns.

The attribute makes a mistaken declaration fail early. It also catches a parent method being removed or renamed while a marked child remains.

Override verifies that a matching inherited method exists (it also supports interface methods, a later topic). It does not require a parent call or verify the method's business logic. Signature compatibility remains a separate PHP rule. Do not put it on `__construct()`. For PHP 8.4 use it on methods; property support belongs to PHP 8.5. See the official Override reference.